

PROJECT.md

MAHABOCW Medical Scholarship Verification Automation

Intelligent Document Processing (IDP) & Browser Automation Pipeline

Project Overview

This project is being developed as part of a consulting engagement involving the Maharashtra Building and Other Construction Workers Welfare Board (MAHABOCW).

The department provides educational financial assistance to the children (or spouse) of registered construction workers. Applicants submit their applications through the MAHABOCW portal along with supporting documents.

The department currently performs a completely manual verification process for every application.

The goal of this project is to automate as much of this verification process as possible while maintaining very high accuracy.

Current Objective

The current task is **NOT** to modify anything inside the government portal.

The only required deliverable is:

Produce an Excel file containing two new columns

```
corrected_college_name  
corrected_college_address
```

These values should contain the official college name and address extracted from the uploaded Bonafide Certificate (and optionally verified using the College ID Card).

The existing Excel sheet should remain unchanged apart from the addition of these new columns.

Dataset

Primary file

Medical_Claim_Data_Part_1.xlsx

Primary worksheet

Medical Data Verification-Par1

Number of records

Approximately **21,348**

Relevant columns

acknowledgement_no
full_name
child_name
academic_years
college_name
college_address
registration_no
aadhar_number
degree_name

The remaining columns are not required for the current automation.

Problem Statement

The dataset suffers from two independent issues.

Problem 1

Duplicate applications.

Some beneficiaries have applied multiple times during the same academic year.

This issue is being solved separately.

It is NOT the focus of this automation.

Problem 2 (Current Focus)

Incorrect or incomplete college names.

Examples

Applicant enters

TAKHATMAL SHRIVALABH HOMOEOPATHIC MEDICAL COLLEGE

Actual college

TAKHATMAL SHRIVALLABH HOMOEOPATHIC MEDICAL COLLEGE & HOSPITAL

Applicant enters address

Amravati

Actual address

Rajapeth, Amravati

The automation should extract the official version from the uploaded documents.

Existing Manual Workflow

The consultant performs the following steps for every single record.

Step 1

Open Excel.

Select an acknowledgement number.

Example

6834821252

Step 2

Open

<https://iwbms.mahabocw.in/sso>

Login manually.

The automation intentionally does NOT automate login.

Step 3

After login the user reaches

<https://iwbms.mahabocw.in/administrator>

This is an Angular Single Page Application (SPA).

Important:

The URL DOES NOT change when navigating between sections.

Step 4

Click

Claims

The browser remains on

/administrator

because the page is an SPA.

Step 5

The claims table appears.

Initially it does NOT contain the acknowledgement number column.

The user manually drags

Acknowledgement Number

into the grid.

This is currently done manually before the automation starts.

Step 6

The user clicks

three horizontal menu bars

on the Acknowledgement Number column.

Then clicks

Filter

(the funnel icon)

This opens the filter textbox.

The textbox remains open.

Automation starts here.

Current Browser Automation

The current Playwright automation successfully performs the following.

For every row in Excel

Read

acknowledgement_no

Example

6834821252

Then

Fill the filter textbox

Click

Apply

Wait for filtering

Click

View Claim Form

A new browser tab opens.

This part is fully working.

Claim Form Page

URL

<https://iwbms.mahabocw.in/claim-management/claim-main-form>

The claim page contains

Worker information



Beneficiary information



Institute Details



Documents Required

The automation currently reaches this page successfully.

Important Sections

The relevant section is

DOCUMENTS REQUIRED

The page contains five uploaded documents.

1.

Bonafide Certificate of Current Academic Year

2.

College Identity Card

3.

Child Aadhaar Card

4.

Ration Card

5.

Self Declaration

Only the first two documents matter for this project.

Manual Verification Process

For every claim

The consultant

Clicks

View Document

beside

Bonafide Certificate

This opens a new browser tab

URL resembles

`https://storage.mahabocw.in/getfile/xxxxxxx.pdf?x-access-token=xxxxxxx`

The PDF is scanned manually.

The consultant verifies

Student Name

Academic Year

Official College Name

Official College Address

Then updates Excel manually.

Initial Automation Attempt

The original strategy attempted to

Click

View Document

using Playwright

Capture the new browser tab

Read the URL

Download the PDF

OCR the first page

This failed.

Root Cause Analysis

After debugging

The following facts were discovered.

Discovery 1

The page loads correctly.

The page contains

DOCUMENTS REQUIRED

Bonafide Certificate

[View Document](#)

Discovery 2

Playwright successfully detects

5

[View Document](#)

links.

Discovery 3

The HTML contains

0

<tr>

elements

The original script searched

tr

↓

Bonafide

↓

View Document

Since there are no table rows

Playwright waited forever.

Eventually timing out.

HTML Structure

The HTML around the Bonafide document looks like

```
<label>

Bonafide Certificate

</label>

<div class="row">

  <input disabled>

  <a href="https://storage.mahabocw.in/getfile/...">

    View Document

  </a>

</div>
```

Important discovery

The

href

already contains the complete authenticated PDF URL.

This means

there is NO need to click

View Document

at all.

Major Design Change

Old pipeline

Playwright

↓

Click View Document

↓

Open New Tab

↓

Read URL

↓

Download PDF

New pipeline

Playwright

↓

Open Claim Form

↓

Read HTML

↓

Extract href



This is much simpler.

It avoids

new tabs

popup timing

click failures

token synchronization

Current Architecture

Stage 1

Portal Automation

Status

Completed

Stage 2

Extract PDF URLs directly from HTML

Status

In Progress

Stage 3

Download PDFs

Status

Pending

Stage 4

OCR

Status

Pending

Stage 5

LLM Information Extraction

Status

Pending

Stage 6

Excel Output

Status

Pending

Planned Technology Stack

Browser Automation

Playwright

HTML Parsing

BeautifulSoup4

PDF Processing

PyMuPDF

OCR

PaddleOCR (GPU)

EasyOCR is currently used temporarily.

It will be replaced.

Image Processing

OpenCV

LLM

Local model

Options

Ollama

or

LM Studio

No paid APIs.

No cloud OCR.

No AWS Textract.

Excel

Pandas

Current Script Status

Working

- ✓ Reads Excel
- ✓ Loads correct worksheet
- ✓ Reads acknowledgement numbers
- ✓ Filters claim grid
- ✓ Opens claim form
- ✓ Waits for page
- ✓ Can read page HTML
- ✓ Can read page text
- ✓ Can detect

[View Document](#)

links

Not Working

- ✗ Clicking Bonafide
- ✗ Clicking College ID

Reason

Incorrect locator strategy.

The page contains no

`<tr>`

elements.

Immediate Next Goal

Implement Stage 2.

Use BeautifulSoup to parse

```
new_page.content()
```

Locate

```
Bonafide Certificate
```

Find the following sibling

```
<div>  
  
↓  
  
<a>  
  
↓  
  
href
```

Extract

Bonafide PDF URL

College ID URL

without clicking anything.

Expected output

```
Bonafide  
  
https://storage.mahabocw.in/getfile/....  
  
College ID
```

```
https://storage.mahabocw.in/getfile/....
```

Once this works

Playwright interaction with documents will be completely eliminated.

Future OCR Pipeline

Once the PDF is downloaded

1.

Render first page

↓

2.

Crop top ~20-25%

↓

3.

Run PaddleOCR (GPU)

↓

4.

Send OCR output to local LLM

↓

5.

Extract

Official College Name

Official Address

Student Name

Academic Year

↓

6.

Compare with Excel values

↓

7.

Write

```
corrected_college_name
```

```
corrected_college_address
```

Hardware

Development Machine

Windows Laptop

GPU

NVIDIA RTX 4060 Laptop GPU

Current issue

EasyOCR is running on CPU.

CUDA acceleration has not yet been configured.

Future OCR will use GPU-enabled PaddleOCR.

Guiding Principles

- Do not automate login.
- Do not modify portal data.
- Output only a corrected Excel workbook.
- Avoid unnecessary browser interactions.
- Prefer HTML parsing over UI clicking whenever possible.
- Build and validate one stage at a time.
- Every stage should be independently testable before moving to the next.
- Minimize brittle selectors; prefer semantic parsing and direct URL extraction where available.